

Práctica Nro. 3

Pruebas avanzadas de API con Postman

Materia	INF780 — Verificación y Validación de Software
Carrera	Ingeniería Informática
Universidad	Universidad Autónoma Tomás Frías
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Modalidad	Individual
Referencia	Guía de Laboratorio Nro. 5

Potosí — Bolivia

COMPETENCIA

Diseña e implementa pruebas avanzadas de API REST con Postman, aplicando filtrado por parámetros de consulta, flujos de prueba encadenados con variables dinámicas, y casos de prueba de diseño propio con justificación técnica.

1. DESCRIPCIÓN

Esta tarea plantea escenarios de prueba **distintos a los trabajados en la Guía de Laboratorio Nro. 5**. La Guía 5 sirve como material de referencia para la sintaxis de **pm.***, la configuración de variables y la estructura de colecciones. El estudiante deberá aplicar esos conocimientos en dos partes con escenarios nuevos sobre el mismo proyecto **Movies API** (NestJS + TypeORM + PostgreSQL).

2. OBJETIVOS

- Implementar pruebas de filtrado mediante query parameters con aserciones sobre el contenido del array de respuesta.
- Diseñar y ejecutar una suite de flujo encadenado que comparta estado entre requests mediante variables de colección.
- Diseñar casos de prueba propios con justificación técnica documentada.

3. CONTEXTO: MOVIES API

El proyecto base es **INF780-MoviesApi** (tag **v1.0**). El recurso principal es **/movies** con los cinco endpoints CRUD ya conocidos. Para esta tarea se asume que la API cuenta además con soporte de **query parameters de filtrado** en el endpoint GET **/movies**. Si el proyecto base no los implementa, el estudiante deberá agregar el filtrado como parte de la tarea (ver sección 3.1).

3.1 Query parameters disponibles (o a implementar)

Parámetro	Tipo	Comportamiento esperado
genre	string	Devuelve solo las películas cuyo genre coincide exactamente con el valor enviado.
year	number	Devuelve solo las películas cuyo year coincide con el valor enviado.
rating	number	Devuelve solo las películas con rating mayor o igual al valor enviado.

NOTA

Si el endpoint **GET /movies** del proyecto base no acepta query parameters, el estudiante debe implementarlos en **movies.service.ts** usando **TypeORM QueryBuilder** antes de redactar los tests.

4. PARTE 1 — PRUEBAS DE FILTRADO POR QUERY PARAMETERS

Crear una carpeta **01 Filtrado Query Params** dentro de la colección. Implementar los 7 casos predefinidos a continuación. Para cada caso, el estudiante debe verificar no solo el código HTTP, sino también el **contenido del array de respuesta**: que todos los elementos cumplan el filtro aplicado.

ID	Request	Escenario	Resultado esperado
P1-01	GET /movies?genre=sci-fi	Filtrar por género válido	200 + array con solo películas sci-fi
P1-02	GET /movies?genre=drama	Filtrar por otro género válido	200 + array con solo películas drama
P1-03	GET /movies?year=2010	Filtrar por año exacto	200 + array con películas del año 2010
P1-04	GET /movies?genre=sci-fi&year=2010	Filtrar combinando género y year	200 + array que cumple ambos filtros
P1-05	GET /movies?genre=musical	Género fuera del enum	200 con array vacío [] o 422
P1-06	GET /movies?year=1800	Año menor al mínimo permitido	200 con [] o 422 según implementación
P1-07	GET /movies?rating=9	Filtrar por rating mínimo	200 + solo películas con rating >= 9

4.1 Aserciones requeridas por caso

Además del status code, cada caso debe incluir las siguientes aserciones sobre el body:

- **P1-01 y P1-02:** Verificar que todos los elementos del array tienen el campo genre igual al valor filtrado.
- **P1-03:** Verificar que todos los elementos del array tienen el campo year igual al valor enviado.
- **P1-04:** Verificar que todos los elementos cumplen simultáneamente el genre y el year del filtro combinado.
- **P1-05 y P1-06:** Verificar el comportamiento real de la API (array vacío o 422); documentar en comentario del request cuál fue y por qué.
- **P1-07:** Verificar que todos los elementos tienen rating mayor o igual a 9.0 (usar parseFloat para comparación numérica).

TIP

Para verificar que todos los elementos de un array cumplen una condición, usar `body.every(m => ...)` dentro de `pm.expect().to.be.true`.

```
pm.test("All movies match the genre filter", function () {
  const body = pm.response.json();
  const genre = pm.environment.get("filterGenre");
  pm.expect(body.every(m => m.genre === genre)).to.be.true;
});
```

5. PARTE 2 — SUITE DE FLUJO ENCADENADO

Crear una carpeta **02 Flujo Encadenado** dentro de la colección. La suite simula el ciclo de vida completo de dos películas (A y B) verificando persistencia, actualizaciones y limpieza de datos. El orden de ejecución es estricto.

5.1 Casos predefinidos (9 casos)

ID	Método	Endpoint	Escenario	Resultado esperado
P2-01	POST	/movies	Crear película A con datos válidos	201 + id guardado en variable
P2-02	POST	/movies	Crear película B con datos válidos	201 + segundo id guardado
P2-03	GET	/movies	Verificar que ambas películas aparecen en el listado	200 + las dos IDs presentes
P2-04	PATCH	/movies/{{idA}}	Actualizar título y rating de película A	200 + campos actualizados
P2-05	GET	/movies/{{idA}}	Confirmar que la actualización persiste	200 + nuevos valores correctos
P2-06	DELETE	/movies/{{idA}}	Eliminar película A	200
P2-07	GET	/movies/{{idA}}	Verificar que película A ya no existe	404
P2-08	GET	/movies	Verificar que película B sigue en el listado	200 + idB presente, idA ausente
P2-09	DELETE	/movies/{{idB}}	Eliminar película B (limpieza)	200

IMPORTANTE

Los ids de película A y B deben guardarse como variables de colección (**idA** e **idB**) en el script Tests de los casos P2-01 y P2-02 respectivamente, y reutilizarse en todos los casos siguientes con **{{idA}}** y **{{idB}}** en la URL.

6. RÚBRICA DE EVALUACIÓN

#	Criterio	Descripción	Puntos
1	Parte 1 — Filtrado por query params	Los 7 casos implementados con aserciones correctas: status code, tipo de respuesta array, y verificación de que cada elemento cumple el filtro aplicado.	50
2	Parte 2 — Suite de flujo encadenado (predefinidos)	Los 9 casos predefinidos ejecutados en orden correcto con variables idA e idB capturadas y reutilizadas. Aserciones de persistencia y limpieza.	50
TOTAL			100